MAGAMED IMAMGULIEV (558950)

Università degli Studi di Messina

Object-Oriented Programming – CHESS Project Report





Chess Game Project Description

The Chess project demonstrates all major Object-Oriented Programming (OOP) principles, including **encapsulation, abstraction, inheritance, and polymorphism**. The game features comprehensive chess logic, such as piece movement, checkmate detection, pawn promotion (with user choice), and special rules like *en passant*. Each chess piece—such as the knight, which moves in an “L” shape, or the queen, which combines diagonal and straight moves—is implemented with its unique logic.

A graphical user interface (GUI) displays the chessboard and pieces, with piece images sourced from reputable web resources like Wikipedia. The project also implements advanced chess mechanics, such as castling between the rook and king. Turn-based logic ensures that only the correct side can move during their turn.

The motivation behind this project was to apply and showcase OOP concepts in a strategic game setting. By building chess, I have thoroughly explored encapsulation (with private fields and accessors), abstraction (through abstract and base classes), inheritance (via specialized piece classes), and polymorphism (overridden movement logic), as described in detail below.

Four packages are used in this chess project:
- image
- main
- Piece
- ai

The **image** package is responsible for illustrating the chess pieces on the board, ensuring each piece is visually represented during gameplay. The **main** package handles the core functionalities of the chess game, including movement logic, input management (such as mouse and touchpad controls), and the play board, which serves as the main interface displaying all actions and updates. The **Piece** package contains all the chess pieces and defines their individual movement rules and characteristics, forming the foundation of the game’s mechanics. Additionally, the **ai** package controls the AI opponent, allowing for single-player gameplay. If desired, the AI can be disabled or removed by on/off button which will be built on the right side of the board.

Movement and Collision Logic for Chess Pieces

Piece	***Move Validation***	***Logic***
Bishop	Diagonal only	Diagonal square check for obstruction
Rook	Horizontal or vertical only	Straight line square check for obstruction
Queen	Combines bishop and rook logic	Combines both diagonal and straight checks
King	One square in any direction, castling	None (1 square) or special castle check
Knight	L-shape movement	None (can jump over pieces)
Pawn	Forward, double first, diagonal capture, en passant	Only checks for destination squares

For my project, I implemented the logic and movement for all chess pieces by using specific rules and collision checks. The bishop moves diagonally, verified by ensuring the absolute difference in its rows and columns is equal, and it checks each diagonal square for possible obstructions. The king can move one square in any direction or perform castling, which includes verifying the rook's position and checking for empty tiles. The knight's movement follows an L-shape pattern that does not require collision checking because it jumps directly to its target square. Pawns can move forward by one square, two squares on their first move, capture diagonally, or perform en passant, depending on the piece in the destination square. The rook moves straight horizontally or vertically, checking each square along the path for obstructions. The queen combines the logic of the rook and bishop, moving either straight or diagonally, and checks for obstructions similarly. In general, all pieces that move in straight or diagonal lines (rook, bishop, queen) use loops to verify whether the path to the target square is clear of other pieces. These movement and collision checks ensure that each piece follows the rules of chess and interacts correctly with other pieces on the board. I chose to implement the movement and collision detection logic for chess pieces because it is a perfect example of OOP. Each chess piece has distinct behavior and movement rules, making it ideal to represent as a class with encapsulated data and methods.

1. Encapsulation

Encapsulation is one of the core principles of object-oriented programming. It refers to the practice of bundling an object's data (its fields) and the methods that operate on that data into a single unit, typically a class, and controlling access to that data. The main goal of encapsulation is to prevent direct access to an object's internal state and instead force interaction through well-defined methods. This makes the codebase more robust, easier to debug, and less prone to unexpected side effects.

In this chess project, encapsulation is clearly demonstrated in the Piece class. All the attributes that represent a piece's state—such as its column and row position, color, name, whether it's the first move, its visual representation (`sprite`), and so on—are declared as private. This ensures that no external class, including its subclasses like Bishop or Rook, can directly modify or access these fields. Instead, the class provides a set of public getter and setter methods, such as `getCol()`, `setCol(int col)`, `isWhite()`, and `setWhite(boolean isWhite)`. These methods act as a controlled interface for interacting with the object's state.

```
private int col, row;
private int xPos, yPos;
private boolean isWhite;
private String name;
private int value;
private boolean isFirstMove = true;
private boolean captured = false;
private BufferedImage sheet;
private int sheetScale;
private Image sprite;
private Board board;
```

```
public int getCol() {
    return col;
}
public void setCol(int col) {
    this.col = col;
}
public int getXPos() {
return xPos;
}
public void setXPos(int xPos) {
this.xPos = xPos;
}
```

For example, the Bishop class uses these methods in its constructor to initialize its state. It sets the piece's color and position using `setCol(col)`, `setRow(row)`, and `setWhite(isWhite)`. These calls do not directly modify the fields of the superclass Piece; instead, they go through the encapsulated interface. This also means that any logic added later inside these setter methods—for instance, bounds checking or move validation—would automatically apply to all subclasses without changing their code.

```
this.setCol(col);
this.setRow(row);
```

```
this.setXPos(col * board.getTileSize());
this.setYPos(row * board.getTileSize());
```

Encapsulation is also visible in the **Board** class. Variables like `tileSize`, `rows`, `cols`, `isGameOver`, and the list of all pieces (`pieceList`) are declared private or final, and accessed via public methods like `getTileSize()`, `isGameOver()`, and `getPieceList()`. This ensures that even classes which rely heavily on the board's structure—like **ChessAI**—must interact with it through a consistent and safe interface. A good example is the way the `selectedPiece` is handled: the board allows outside code to get or set the currently selected piece only through the methods `getSelectedPiece()` and `setSelectedPiece(Piece p)`, never by directly accessing the field.

```
private static final int tileSize = 85;

private static final int cols = 8;

private static final int rows = 8;

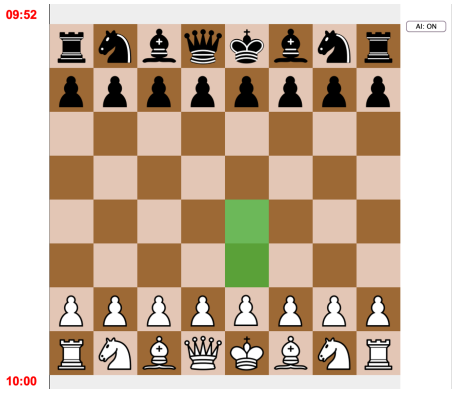

public int getTileSize() { return tileSize; }

public int getCols() { return cols; }

public int getRows() { return rows; }
```

These constants are **private** but exposed safely through **public** getter methods. This allows other classes to read their values without being able to change them.

Moreover, encapsulation provides great flexibility for future changes. For instance, if later you decide to change the way a piece's position is stored (perhaps from row/column to a single integer), you could do so without needing to modify all the classes that use `getCol()` and `getRow()`, because those classes are already encapsulated from the internal implementation. This modular approach makes the software more maintainable and helps prevent bugs caused by direct access or inconsistent updates.



User-selected white pawn: The game internally accesses and updates the pawn's position through encapsulated methods like `getCol()`, `getRow()`, and `setSelectedPiece(Piece)` — not by directly modifying private fields. The green tile highlights the selected piece's legal move area, calculated using `isValidMovement()` and internal position getters.

2. Information Hiding

While encapsulation is primarily a programming technique used to restrict access to class fields, information hiding is a broader design strategy. It is about deliberately concealing aspects of a class or module that are not essential to its interface, thus reducing the complexity seen by users of that class. The purpose of information hiding is to isolate changes and ensure that only what is strictly necessary is exposed to the rest of the program.

In this project, the principle of information hiding is applied consistently across the **Board**, **Piece**, and **ChessAI** classes. A particularly strong example is how the internal turn management is handled in the Board class. The variable `isWhiteToMove` is declared as private, and the only way to check or change the current player's turn is by calling `isWhiteToMove()` and `setIsWhiteToMove(boolean turn)`. External classes, such as **ChessAI**, do not need to know how this state is stored or managed—they only interact with it through a simple, clean interface. As a result, if the internal logic for handling turns were ever changed, the external classes would remain unaffected.

```
private boolean isWhiteToMove = true;
```

```
public boolean isWhiteToMove() {

return isWhiteToMove;

}

public void setIsWhiteToMove(boolean turn) {

this.isWhiteToMove = turn;

}
```



- The `isWhiteToMove` variable is never accessed directly from external classes like **ChessAI**.
- All reads and updates happen through well-defined methods — hiding how the turn system works.

Similarly, information hiding is used in managing the chess timers for both players. The fields `whiteTimer` and `blackTimer` are declared private and are only manipulated internally through methods such as `setGameOver(boolean over)` or via the constructor. The Board class takes full responsibility for starting, stopping, and updating the timers. From the point of view of an external class, like a GUI handler or AI module, there is no need to understand how the timers are implemented—only that calling a method like `setGameOver(true)` will appropriately stop both timers and end the game. This abstraction simplifies the external code and ensures that the timers are always used correctly.

```
private ChessTimer whiteTimer;

private ChessTimer blackTimer;
```

- **Board** controls the timers and their behavior.

```
public void setGameOver(boolean over) {
    this.isGameOver = over;
    if (over) {
        whiteTimer.stop();
        blackTimer.stop();
    }
}
```

- Other classes can trigger `setGameOver(true)` but don't need to know how the timers stop or function internally.

Furthermore, information hiding helps keep the rules of the game encapsulated. For instance, logic for special moves such as castling, pawn promotion, or en passant captures are all contained within `moveKing()`, `movePawn()`, or `promotePawnTo()` methods in Board. The client code such as the mouse event handlers or AI algorithm does not need to know how these rules are implemented. They simply call `makeMove(Move move)` and trust that all special handling will be performed as necessary. This clear separation between interface and implementation is a hallmark of well-designed information hiding.

Information hiding leads to more maintainable and scalable code. By reducing dependencies between classes, it ensures that internal implementation details can change without widespread modifications across the entire codebase. It also improves clarity and usability, as classes present a minimal, understandable interface to other parts of the program.



This interface state (timers stopped, game frozen) is achieved simply by calling `setGameOver(true)`. External classes like AI or mouse input don't need to know that this method stops both timers — internal details are hidden behind a clean interface.

3. Abstraction

The definition of abstraction is showing only essential features of a class and hiding unnecessary implementation details from the user. It allows to work with objects at a high level, without knowing how they work inside. The implementation as follows:

An interface in Java defines a set of method signatures (what a timer can do) but provides no code for how to do it. It acts as a **contract**—any class that claims to be a GameTimer must implement all these methods.

1. Interface (GameTimer) – the pure abstraction (The contract):

```
public interface GameTimer {
    void start();
    void stop();
    int getTimeLeft();
    void setTimeLeft(int timeLeft);
    boolean isRunning();
    void setRunning(boolean running);
    String getTimeText();}
```

2. Abstract Class (AbstractTimer) — Partial Abstraction and Code Sharing

An abstract class implements the interface (the contract) but only provides code for some of the methods.

```
public abstract class AbstractTimer implements GameTimer {
    protected int timeLeft;
    protected boolean isRunning;
    public AbstractTimer(int startSeconds) {
        this.timeLeft = startSeconds;
        this.isRunning = false;
    }
    @Override
    public abstract void start();
    @Override
    public abstract void stop();}
```

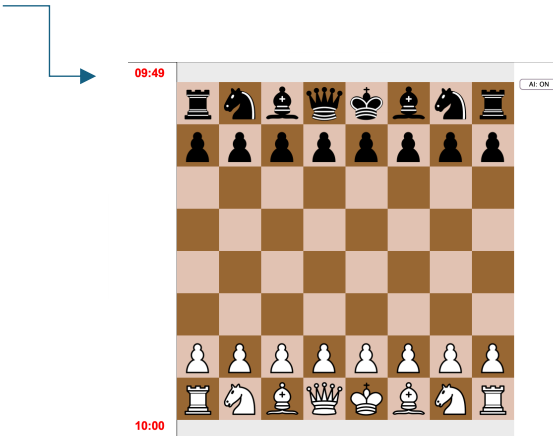
3. Concrete Class (ChessTimer) — Implementation (The Real Thing)

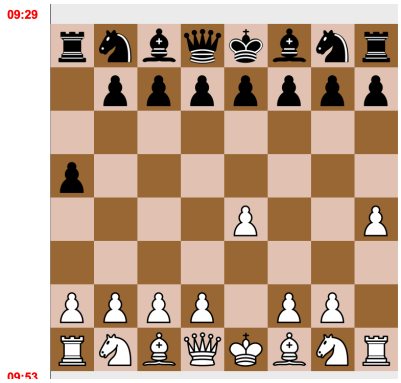
A regular class that extends the abstract class and implements all the required behaviors. Provides the actual code (the “how”) for things like `start()` and `stop()`, possibly using libraries and logic specific to this timer type.

```
public class ChessTimer extends AbstractTimer {
    // ...
    @Override
    public void start() {
        // Starts the timer
    }
    @Override
    public void stop() {
        // Stops the timer
    }
    // ... timer-specific code ...
}
```

How they work together (The power of Abstraction)

- Code elsewhere can refer to a timer by the interface (**GameTimer**), not caring about which specific timer is used.
- **AbstractTimer** reduces code duplication and ensures all timers have shared logic (like formatting time).
- **ChessTimer** provides the actual behavior, using the abstract “template” above.





3. Inheritance

Inheritance is a fundamental concept where a class (**child** or **subclass**) automatically gains the properties and behaviors (fields and methods) of another class (**parent** or **superclass**).

The implementations happens where child class automatically gets all the variables and methods of parent class and child class can add new methods/variables or override the ones from parent class. It also enables code reuse and helps model real- world relationship. In the project, this property applied via Piece.java parent class (superclass). All other specific pieces are Children (subclasses) for example: Queen, King, Rook, Bishop, Knight, Pawn.

Declaration implementation example:

```
public class Queen extends Piece { ... }

public class King extends Piece { ... }

public class Rook extends Piece { ... }
```

Each subclass inherits fields, methods and any logic or variables declared in Piece.java.

- Fields: e.g. col, row, `isWhite`, etc.
- Methods: e.g. `getCol()`, `setCol()`, `getBoard()`, etc.
- Any logic or variables declared in Piece can be used in all these subclasses without rewriting them.

```
public class Piece {

private int col, row;

// ...getters/setters, paint(),
etc.

}
```



PARENT (SUPERCLASS)

```
public class Knight extends Piece {

    public Knight(Board board, int col, int row,
boolean isWhite) {

        super(board); // Calls constructor

        this.setCol(col);

    }

    @Override

    public boolean isValidMovement(int col, int
row) {

        // Custom logic for Knight

    }

}
```



CHILD (SUBCLASS)

4. Polymorphism

Polymorphism in object-oriented programming refers to the ability of a single interface or superclass to represent different types of behavior in subclasses. It allows the same method name to behave differently depending on the object that is using it. Java supports different forms of polymorphism, including:

- Polymorphism (dynamic binding)
- Parametric polymorphism
- Coercion

Dynamic Binding occurs when a method defined in a superclass (Piece) is **overridden** by its subclasses (Bishop, Queen, etc.), and the decision of which method to execute is made **at runtime**. This is demonstrated when the movement validation of a piece is handled through the superclass reference.

- Superclass method (in Piece.java):

```
public boolean isValidMovement(int col, int row) {

     return true;

}
```

- Overriden in subclass (in Bishop.java):

```
@Override

public boolean isValidMovement(int col, int row) {

    return Math.abs(this.getCol() - col) == Math.abs(this.getRow() -
row);

}
```

- Client usage with dynamic binding (in Input.java):

```
Piece pieceXY = board.getPiece(col, row);

...

Move move = new Move(board, board.getSelectedPiece(), col, row);

if (board.isValidMove(move)) {

…

}
```

Although piece is referenced as a Piece, the overridden isValidMovement method of the actual subclass (e.g., Bishop) is invoked at runtime.

Parametric Polymorphism, this is when the code is written to operate on a type parameter and can work with objects of various types. Java achieves this using generic. The project uses this with collections:

- Board.java

```
private final ArrayList<Piece> pieceList = new ArrayList<>();
```

The `ArrayList<T>` is a generic class that allows polymorphic usage. In your case, it's specifically used as `ArrayList<Piece>`, allowing storage of multiple Piece subclasses like Bishop, Queen, etc.

Coercion polymorphism happens when an object of one type is **automatically converted** to another compatible type (e.g., subclass to superclass or interface). This is present in project's timer system.

- Board.java

```
private GameTimer whiteTimer;
private GameTimer blackTimer;
```

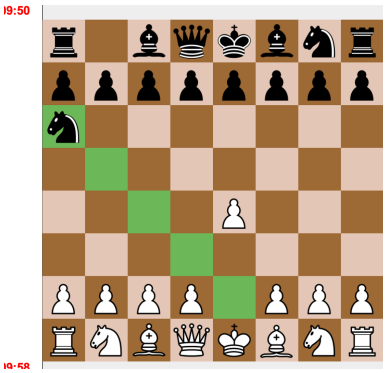
This allows the variables to hold any object that implements the `GameTimer` interface — in this case, `ChessTimer`.

- Board.java

```
whiteTimer = new ChessTimer(startSeconds, () -> {
    handleGameOver("Black wins by time!");
}, this::repaint);
```

ChessTimer is **automatically coerced** into GameTimer.

```
blackTimer = new ChessTimer(startSeconds, () -> {
    handleGameOver("White wins by time!");
}, this::repaint);
```



Although this piece is referenced as a Piece, the diagonal move validation logic is dynamically dispatched to the overridden `isValidMovement()` in the Bishop subclass at runtime. This is classic dynamic polymorphism.

5. Subtyping and Multityping

Subtyping is a principle of object-oriented programming where a subclass object can be treated as an instance of its superclass. This allows you to write generic code that works with the superclass type, while still operating on objects of more specific subclasses.

In chess project, Bishop, King, Rook, Pawn, etc., all **extend the abstract superclass** Piece. This allows the Board, Move, and AI components to work with different pieces polymorphically.

- **Bishop** is a subtype of **Piece**



This means a Bishop **is-a** Piece

```
public class Bishop extends Piece {
    ...
}
```

- Subtype objects stored as **Superclass**

```
pieceList.add(new Bishop(this, 2, 0, false));
pieceList.add(new King(this, 4, 0, false));
```



These are objects of type **Bishop** and **King**, but they are added into an `ArrayList<Piece>` — they are treated as instances of the Piece superclass.

In this project, an object like **Bishop** can be treated as an instance of multiple types due to inheritance, demonstrating the concept of multityping. A **Bishop** is first and foremost an instance of its own class, meaning it can use specific behaviors such as its custom implementation of `isValidMovement(...)`. However, because **Bishop** extends the abstract superclass **Piece**, it is also a **Piece** — and can be accessed through any variable or method expecting a **Piece** object. This is exactly what happens in the **Move** class, where a **Piece** parameter is passed into the constructor and may represent any specific subclass such as a **Bishop**, **King**, or **Pawn**. Therefore, a single **Bishop** object is multi-typed: it is a **Bishop**, a **Piece**, and a **movable entity within the game**.

- Used as **Piece** in Other Classes

```
public class Move {
    Piece piece;
    public Move(Board board, Piece piece, int newCol, int newRow){
        this.piece = piece; // could be any subclass of Piece
    }
}
```

6. Composition

Composition is an object-oriented design principle where a class contains references to other objects as part of its internal structure, allowing one class to use the functionalities of another by "having" it, rather than "being" it (which would be inheritance).

In your chess project, composition is used extensively to model the relationships between game components. One key example is the way the Piece class holds a reference to the Board it belongs to:

- Example 1: **Piece** has-a **Board**

```
private Board board;
//
public Piece(Board board) {
    this.board = board;
    ...
}
```

Each chess piece is created with a reference to the board it belongs to. This allows the piece to interact with the board (e.g., checking if a square is occupied, verifying move legality, or fetching another piece), without inheriting from the Board class — demonstrating **composition over inheritance**.

- Example 2: **Move** has-a **Piece**

```
private Piece piece;
private Piece capture;


public Move(Board board, Piece piece, int newCol, int newRow) {
    ...
    this.piece = piece;
    this.capture = board.getPiece(newCol, newRow);
}
```

The Move class represents a single movement on the board and **contains** (has-a) both the piece being moved, and the piece (if any) being captured. This composition allows Move to encapsulate complete information about the transition.

7. Extensibility

In **Object-Oriented Programming (OOP)**, **extensibility** refers to the ability of a software system to be **easily extended** with new functionality **without modifying existing code**. This makes programs more maintainable, scalable, and adaptable to future changes.

From my personal perspective, extensibility should be supported by the following principles:

- **Abstraction** – Define general contracts (e.g., abstract classes or interfaces) that hide implementation details and allow variation underneath.
- **Inheritance** – Extend base classes to create specialized behavior in subclasses.
- **Polymorphism** – Write code that uses base types (e.g., Piece), so that it can work with any subclass (e.g., Bishop, Queen) without change.
- **Composition** – Combine objects to build more complex functionality without modifying their internals.

The Piece class defines common properties and behavior for all chess pieces (e.g., position, sprite, board reference), but it defers the actual movement logic to the subclasses via the isValidMovement() method. This makes Piece a **template** or **contract** for all future pieces. Any new subclass of Piece can override isValidMovement() and define its own rules. For example, to add a hypothetical new piece like **Matador** (which moves like both a queen and a knight):

```
public class Matador extends Piece {

    @Override

    public boolean isValidMovement(int col, int row) {

        // Combine Queen and Knight movement logic

    }

}
```

Extensibility is also closely tied to the Open-Closed Principle which is one of the core principles of SOLID modular rules. All the classes should be open for extension but closed for modification which means that the creator must be able to extend system behaviour by adding new code (subclasses or components), not by changing the old code. Additionally, this project also obeys single responsibility principle. For example, Piece package contains pieces such as Bishop, King, Knight, and etc. and Piece. Each class uniquely complete own job. For example, Bishop has its own logic of movement.